Task 3: Image Negative Transformation

Applies intensity inversion on a grayscale image by subtracting each individual pixel intensity from 255.

1. এই প্রোগ্রামটির মূল Concept কী?

এই প্রোগ্রামটির মূল কাজ হলো একটি Grayscale Image-কে Negative Image-এ Convert করা।

Image Negative বলতে বোঝায় ছবির প্রতিটি Pixel-এর Intensity Value উল্টে দেওয়া। অর্থাৎ, ছবির কালো অংশ সাদা হয়ে যাবে এবং সাদা অংশ কালো হয়ে যাবে। একইভাবে Gray Pixel-গুলোর Intensity-ও উল্টে যাবে।

উদাহরণ:

- Black (0) → White (255)
- White (255) → Black (0)
- Gray (100) → Gray (155)
- Gray (200) → Gray (55)

এই কাজটি করার জন্য আমরা একটি Mathematical Formula ব্যবহার করব।

2. Mathematical Formula

Image Negative করার Formula হলো:

$$Negative = 255 - Original$$

এখানে:

- Original = Original Pixel Intensity
- Negative = New Pixel Intensity
- 255 = Maximum Intensity Value (8-bit Image)

উদাহরণ:

ধরি, একটি Pixel-এর Original Intensity Value 80।

তাহলে:

$$Negative = 255 - 80$$

$$Negative = 175$$

অর্থাৎ, Original Pixel-এর Intensity ছিল 80, Negative করার পর সেটি 175 হয়ে যাবে।

3. সম্পূর্ণ Code (বাংলা Comment সহ)

```
from PIL import Image


image = Image.open("PNG/Task_03_Image_Negative.png")


if image.mode != "L":
   raise ValueError("Image must be in grayscale mode (L)")
width, height = image.size
pixels = image.load()
for y in range(height):
   for x in range(width):
      original_pixels_intensity = pixels[x, y]
      negative_pixels_intensity = 255 - original_pixels_intensity
      pixels[x, y] = negative_pixels_intensity


image.save("Output/Task_03_Image_Negative_negative.png")
print("Negative image saved as 'Output/Task_03_Image_Negative_negative.png'")
```

4. Step-by-Step Code Explanation

Step 1: Import Library

```
from PIL import Image
```

এখানে আমরা Python-এর Pillow Library থেকে Image Module Import করেছি।

এই Library ব্যবহার করে Image Open, Process এবং Save করতে পারি।

Step 2: Open the Image

```
image = Image.open("PNG/Task_03_Image_Negative.png")
```

এই লাইনে আমরা নির্দিষ্ট Location থেকে Original Image Open করছি।

এখানে একটি বিষয় মনে রাখতে হবে, এই প্রোগ্রামটি Grayscale Image নিয়ে কাজ করবে।

Step 3: Check Image Mode

if image.mode != "L": raise ValueError("Image must be in grayscale mode (L)")

এটি এই প্রোগ্রামের একটি গুরুত্বপূর্ণ অংশ।

এখানে image.mode ব্যবহার করে ছবিটি কোন Format-এ আছে তা পরীক্ষা করছি।

"L" দ্বারা 8-bit Grayscale Image বোঝায়।

যদি Image Grayscale Mode-এ না থাকে, তাহলে raise ValueError() ব্যবহার করে একটি Error দেখাবে এবং প্রোগ্রাম বন্ধ হয়ে যাবে।

এর ফলে ভুল Format-এর Image নিয়ে Negative Operation করা হবে না।

Step 4: Get Image Dimensions

width, height = image.size

এখানে Original Image-এর Width এবং Height বের করছি।

Width হলো ছবির Horizontal Pixel সংখ্যা এবং Height হলো Vertical Pixel সংখ্যা।

এই দুটি Value ব্যবহার করে ছবির প্রতিটি Pixel Access করব।

Step 5: Load Pixel Data

pixels = image.load()

এখানে load() Function ব্যবহার করে Image-এর Pixel Data Access করছি।

এর মাধ্যমে আমরা ছবির নির্দিষ্ট Coordinate-এর Pixel Value পড়তে এবং পরিবর্তন করতে পারব।

যেমন:

pixels[x, y]

এটি নির্দিষ্ট (x, y) Position-এর Pixel Intensity Value প্রদান করে।

Step 6: Traverse Every Pixel

for y in range(height): for x in range(width):

এখানে আমরা দুটি Nested Loop ব্যবহার করেছি।

প্রথম Loop ছবির প্রতিটি Row বরাবর কাজ করে।

দ্বিতীয় Loop প্রতিটি Row-এর প্রতিটি Column-এর Pixel Access করে।

এভাবে ছবির প্রতিটি Pixel একে একে Process করা হয়।

Step 7: Get Original Pixel Intensity

original_pixels_intensity = pixels[x, y]

এখানে pixels[x, y] ব্যবহার করে নির্দিষ্ট Pixel-এর Original Intensity Value সংগ্রহ করছি।

যেহেতু এটি একটি Grayscale Image, তাই প্রতিটি Pixel-এর জন্য একটি মাত্র Intensity Value পাওয়া যাবে।

এরপর Value-টি original_pixels_intensity Variable-এ সংরক্ষণ করছি।

Step 8: Calculate Negative Intensity

negative_pixels_intensity = 255 - original_pixels_intensity

এটি এই প্রোগ্রামের সবচেয়ে গুরুত্বপূর্ণ অংশ।

এখানে Negative Transformation Formula ব্যবহার করছি।

প্রতিটি Pixel-এর Original Intensity Value থেকে 255 বিয়োগ করছি।

এর ফলে Pixel-এর Intensity উল্টে যাচ্ছে।

যেমন, Original Value 50 হলে Negative Value হবে 205।

Step 9: Update Pixel Value

pixels[x, y] = negative_pixels_intensity

এখানে Calculated Negative Value-টি Original Image-এর একই Coordinate-এ বসিয়ে দিচ্ছি।

অর্থাৎ, প্রতিটি Pixel-এর Original Value পরিবর্তন করে Negative Value বসানো হচ্ছে।

এখানে নতুন Image তৈরি করা হয়নি। Original Image-এর Pixel-গুলো সরাসরি পরিবর্তন করা হয়েছে।

Step 10: Save the Negative Image

image.save("Output/Task_03_Image_Negative_negative.png")

সব Pixel Process করা শেষ হলে এই লাইনের মাধ্যমে Negative Image-টি Output Folder-এ Save করছি।

print("Negative image saved successfully!")

সবশেষে Image সফলভাবে Save হয়েছে এমন একটি Confirmation Message প্রদর্শন করছি।

Oral Explanation (বাংলা)

প্রথমে আমি PIL Library Import করেছি, যেটি Python-এ Image Processing-এর জন্য ব্যবহার করা হয়।

এরপর একটি Grayscale Image Open করেছি। তারপর Image-এর Mode পরীক্ষা করেছি, যাতে নিশ্চিত হওয়া যায় যে Image-টি Grayscale Format-এ আছে। কারণ এই প্রোগ্রামটি শুধুমাত্র Grayscale Image-এর জন্য কাজ করবে।

এরপর ছবিটির Width এবং Height বের করেছি এবং load() Function ব্যবহার করে Pixel Data Access করেছি।

তারপর দুটি Nested Loop ব্যবহার করেছি, যাতে ছবির প্রতিটি Pixel একে একে Access করা যায়।

প্রতিটি Pixel থেকে Original Intensity Value সংগ্রহ করেছি। এরপর 255 থেকে Original Intensity Value বিয়োগ করে Negative Intensity Calculate করেছি।

তারপর Calculated Negative Value-টি একই Pixel Position-এ বসিয়েছি।

এভাবে ছবির প্রতিটি Pixel-এর Intensity পরিবর্তন করেছি।

সবশেষে Negative Image-টি Output Folder-এ Save করেছি।

এভাবেই একটি Grayscale Image-কে Negative Image-এ Convert করা হয়েছে।

6. Viva Questions & Answers

Q1. What is an Image Negative?

Ans: Image Negative is an image processing technique that reverses the intensity values of an image.

Q2. What is the formula for Image Negative?

Ans:

\[Negative = 255 - Original \]

Q3. Why do we use 255?

Ans: Because 255 is the maximum intensity value of an 8-bit grayscale image.

Q4. What is the purpose of image.load()?

Ans: It provides access to individual pixel values of an image.

Q5. What is the purpose of pixels[x, y]?

Ans: It accesses the intensity value of a pixel at a specific coordinate.

Q6. Why do we use Nested Loops?

Ans: To access and process every pixel in the image.

Q7. What does "L" mean?

Ans: "L" represents an 8-bit grayscale image with intensity values ranging from 0 to 255.

Q8. Why do we use raise ValueError()?

Ans: To stop the program and display an error if the input image is not in grayscale mode.

Q9. Does this program create a new image?

Ans: No. It modifies the original image in memory and saves the modified image under a new filename.

Q10. What happens to black and white pixels after negative transformation?

Ans: Black becomes white, and white becomes black.

7. গুরুত্বপূর্ণ Functions মনে রাখবে

- Image.open() — Image Open করে।
- image.mode — Image-এর Color Mode পরীক্ষা করে।
- raise ValueError() — Error তৈরি করে।
- image.size — Width এবং Height বের করে।
- image.load() — Pixel Data Access করে।
- pixels[x, y] — নির্দিষ্ট Pixel-এর Value পড়ে বা পরিবর্তন করে।
- image.save() — Image Save করে।